

TAWALA CORE ARCHITECTURE

Version: 1.0.0

Status: Objective Architecture Proposal

Purpose: Define the business and technical foundation of Tawala independently of any existing implementation.

1. What Tawala Is

Tawala is a **multi-tenant, multi-industry business management platform**.

It is designed to help businesses run their operations from one place.

Tawala is not fundamentally a POS.

It is not fundamentally a CRM.

It is not fundamentally an accounting application.

It is a platform that can provide all of these capabilities through one connected business system.

A business may use Tawala for:

- Sales
- Invoicing
- Inventory
- Customers
- Suppliers
- Purchasing
- Payments
- Accounting
- Reporting
- Staff management
- Branch management
- Loyalty
- Appointments
- Industry-specific operations
- Automation
- Integrations

The exact capabilities enabled depend on the needs of the business.

2. The Central Idea

The most important architectural principle is:

Tawala Core provides generic business capabilities. It does not care what industry the business belongs to.

Tawala should be able to support:

- A pharmacy
- A hardware shop
- A supermarket
- A restaurant
- A kinyozi
- A salon
- A wholesaler
- A distributor
- A professional service business
- A SACCO
- And other businesses that can be represented using Tawala's capabilities

The Core should not contain assumptions such as:

```
if business == pharmacy:  
    do pharmacy thing
```

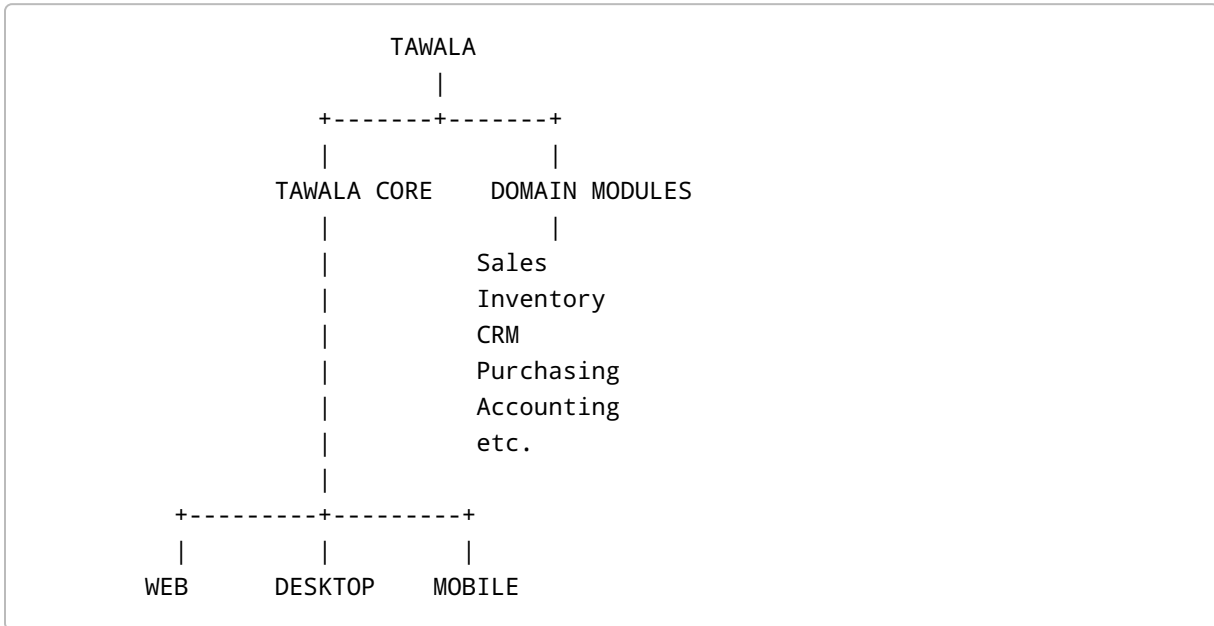
Instead, it provides generic concepts such as:

```
Business  
Party  
Product  
Service  
Location  
Transaction  
Payment  
User  
Role  
Document  
Resource  
Event
```

Industries and specialized capabilities build on those foundations.

3. The Tawala Platform Model

At the highest level:



The Core and domain modules represent the business system.

Web, Desktop, and Mobile are different ways of interacting with that system.

They are not separate versions of the business logic.

4. One Tawala, Multiple Applications

Tawala will have three primary client experiences.

4.1 Desktop

The Desktop application is the primary operational application.

It is designed for businesses that need:

- Fast POS operations
- Barcode scanners
- Receipt printers
- Cash drawers
- Label printers

- Other local hardware
- Potential offline operation
- Full business management

The Desktop application may expose the largest set of operational capabilities.

However:

The Desktop application does not own the business logic.

It communicates with the Tawala platform.

4.2 Web

The Web application provides browser-based access to Tawala.

It is particularly useful for:

- Business setup
- Configuration
- Product management
- User management
- Branch management
- Reports
- CRM
- Accounting
- Administration
- Quick access from any computer

The Web application can expose many of the same capabilities as Desktop, but its experience can be optimized for browser-based work.

4.3 Mobile

The Mobile application is optimized for remote management and quick operations.

Examples:

- Checking sales
- Checking stock
- Viewing business performance
- Monitoring branches
- Viewing customer information
- Approving actions

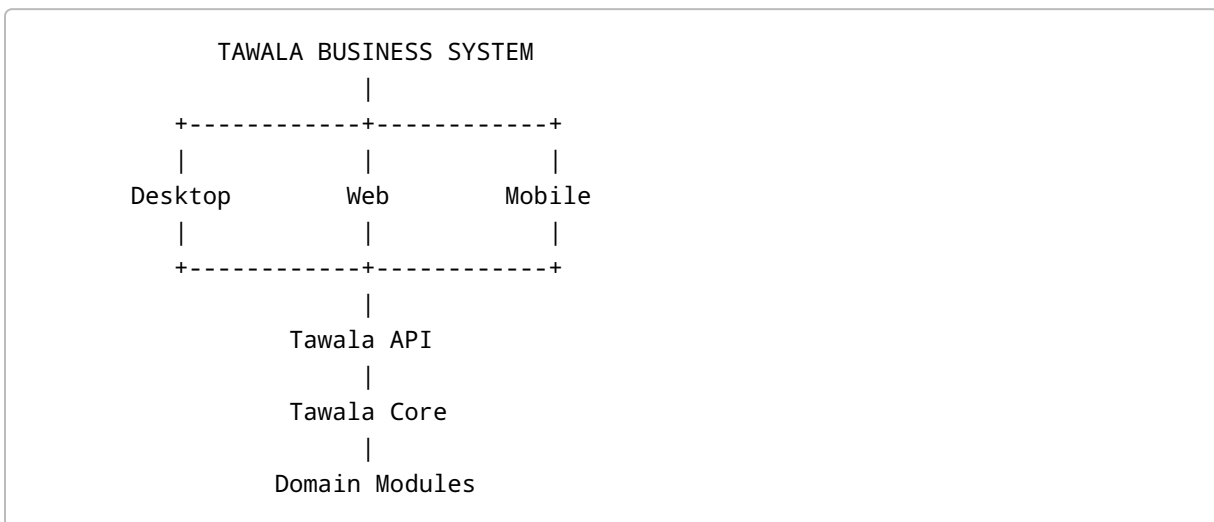
- Receiving alerts
- Managing selected business operations remotely

Mobile does not need to reproduce the entire Desktop experience.

The goal is to provide the right capabilities for the context in which a phone is used.

5. One Business System

The three clients must ultimately operate on the same business model.



A sale created on Desktop must be the same kind of sale when viewed from Web or Mobile.

A customer created on Mobile must be visible from Desktop.

An inventory transfer created on Web must be visible everywhere.

There should never be:

Desktop customer
Web customer
Mobile customer

There should be one:

Customer

with multiple interfaces.

6. Multi-Tenancy

Tawala is fundamentally multi-tenant.

Each business operates inside its own isolated business environment.

A tenant represents an independent business context.

Example:

```
Tawala Platform
|
+-- Business A
|   +-- Branch Nairobi
|   +-- Branch Embu
|
+-- Business B
|   +-- Branch Nairobi
|
+-- Business C
    +-- Branch Meru
```

Business A must never be able to access Business B's data.

This isolation applies to:

- Customers
- Products
- Sales
- Inventory
- Employees
- Suppliers
- Payments
- Reports
- Documents
- Files
- Audit records
- Configuration
- Business settings

Tenant isolation is a **fundamental security requirement**, not an optional feature.

7. Users and Multiple Businesses

A user is not necessarily tied to one business.

One person may participate in multiple businesses.

Example:

```
David
|
+-- Business A
|   Industry: Pharmacy
|
+-- Business B
|   Industry: Kinyozi
|
+-- Business C
    Industry: Hardware
```

The user may have completely different permissions in each business.

For example:

```
David
|
+-- Pharmacy
|   Owner
|
+-- Kinyozi
|   Manager
|
+-- Hardware
    Accountant
```

The user's identity is global.

Their membership and permissions are business-specific.

8. Business Structure

A business can contain multiple operational locations.

```
Business
|
+-- Branch
|   +-- Locations
|   +-- Users
|   +-- Registers
|
+-- Branch
|   +-- Locations
|   +-- Users
|
+-- Branch
```

The exact structure can evolve, but the architecture must support:

- One business with one location
- One business with many branches
- Multiple warehouses
- Multiple stock locations
- Different operational locations within a branch

9. Industry Is Not the Core Boundary

Tawala should not create completely separate foundations for:

```
Pharmacy
Kinyozi
Hardware
Restaurant
SACCO
```

Instead, businesses use combinations of capabilities.

For example:

Hardware business

```
Sales
Inventory
Purchasing
Suppliers
Customers
```

Credit
Accounting

Kinyozi

Customers
Services
Appointments
Employees
Payments
Expenses
CRM

Pharmacy

Products
Inventory
Purchasing
Suppliers
Customers
Sales
Payments
Expiry tracking
Batch tracking

SACCO

Members
Accounts
Contributions
Loans
Repayments
Payments
Accounting
Reporting

The generic Core supports the foundation.

Specialized domains add specialized behavior.

10. Capabilities

A business should be able to use the capabilities it needs.

Potential capabilities include:

```
Commerce
CRM
Inventory
Purchasing
Accounting
Payments
Loyalty
Appointments
Reporting
Staff Management
Automation
```

A business does not need to use everything.

For example:

```
Kinyozi
|
+-- CRM
+-- Services
+-- Appointments
+-- Payments
+-- Reporting
```

while:

```
Hardware Shop
|
+-- Commerce
+-- Inventory
+-- Purchasing
+-- CRM
+-- Accounting
+-- Payments
+-- Reporting
```

Capabilities should be composable rather than forcing every business into the same workflow.

11. Universal Domain Concepts

The Core should contain concepts that make sense across many types of businesses.

The most important candidates are:

Business
User
Membership
Role
Permission
Branch
Location
Party
Product
Service
Resource
Document
Transaction
Payment
Account
Event
Audit Record
Configuration

These concepts form the vocabulary of Tawala.

12. Party

A Party represents an entity that participates in business relationships.

A Party may represent:

- A person
- A company
- A customer
- A supplier
- A member
- An employee
- Another organization

A party can have different relationships with a business.

For example:

```
ABC Hardware Ltd
|
+-- Customer
|
+-- Supplier
```

This prevents Tawala from creating unnecessary duplicate identities.

The business relationship determines what the party can do.

13. Product

A Product represents something the business sells, buys, stores, or otherwise manages.

A product may have:

- Name
- SKU
- Barcode
- Category
- Unit
- Price
- Cost
- Tax information
- Inventory configuration
- Supplier relationships

The Core should not assume that every product behaves identically.

Some businesses may need:

- Batches
- Expiry dates
- Serial numbers
- Multiple units
- Variants

Those should be capabilities or extensions rather than assumptions built into every product.

14. Service

Not every business sells physical products.

A Service represents something delivered to a customer without necessarily being a stocked physical item.

Examples:

```
Haircut
Consultation
Repair
Installation
Cleaning
Training
```

This allows Tawala to support service businesses without pretending that every business is a retail store.

15. Location

A Location represents where something happens or is stored.

Examples:

```
Branch
Warehouse
Shop floor
Store room
Office
Service location
```

Inventory should not simply be treated as one number attached to a product.

The system should be able to understand where stock exists.

Example:

```
Product X

Nairobi Warehouse: 100
```

Nairobi Shop: 20
Embu Warehouse: 50

This becomes important for:

- Branches
- Transfers
- Warehouses
- Stock counts
- Reporting
- Reordering

16. Resource

A Resource represents something a business uses to perform work.

Examples:

Employee
Chair
Room
Machine
Vehicle
Equipment
Work station

Not every business will use the same resources.

The Core provides the concept.

A domain can decide how the resource is used.

For example, a salon may use:

Employee + Chair + Appointment

A repair shop may use:

Employee + Workstation + Job

17. Documents

Business operations frequently produce documents.

Examples:

```
Quote
Sales Order
Invoice
Receipt
Purchase Order
Goods Received Note
Credit Note
Debit Note
Return
```

Documents may have states.

For example:

```
Draft
|
Confirmed
|
Completed
|
Cancelled
```

Not every document needs the exact same lifecycle.

The Core provides common document principles while individual domains define their actual workflows.

18. Transactions

A transaction represents a meaningful business action.

Examples:

```
Sale
Purchase
Return
```

```
Stock Transfer
Payment
Refund
Expense
```

Transactions are important because they create consequences.

For example:

```
Sale
|
+-- Customer relationship changes
+-- Inventory changes
+-- Money changes
+-- Accounting changes
+-- Reports change
+-- Audit record created
```

A transaction should therefore not be treated as merely a database row.

It represents something that happened in the business.

19. Payments

Payment should be a generic concept.

Tawala may support:

```
Cash
M-PESA
Airtel Money
Bank
Card
Credit
Other payment methods
```

The Core should not make M-PESA the definition of payment.

Instead:

```
Payment
|
+-- Method
+-- Amount
+-- Reference
+-- Party
+-- Business
+-- Transaction
+-- Status
```

Specific payment integrations can plug into this model.

20. Accounting

Tawala should eventually support serious financial tracking.

The architecture should allow the system to understand:

- Income
- Expenses
- Accounts receivable
- Accounts payable
- Payments
- Refunds
- Revenue
- Cost of goods sold
- Profit
- Cash flow
- Financial reporting

If full accounting is adopted, it should be based on proper accounting principles rather than simply adding and subtracting values from sales tables.

The accounting domain should be able to consume business events and produce appropriate financial records.

21. Domain Events

Business actions can create events.

Example:

Sale Completed

This may cause:

Inventory updated
Customer history updated
Payment recorded
Accounting updated
Loyalty updated
Reports updated
Audit created

The important idea is:

One business action can have many consequences.

The component that performs the action should not need to manually understand every future consumer of that action.

Example:

```
SaleCompleted
|
+-- Inventory
+-- Accounting
+-- CRM
+-- Loyalty
+-- Reporting
+-- Audit
```

This makes the system easier to extend.

22. Audit Trail

Tawala must know what happened inside the business.

Important actions should record:

Who
What

```
When
Where
Which record
Previous state
New state
Reason
Related transaction
```

For example:

```
Jane
cancelled Invoice INV-1032
at Embu Branch
at 14:32

Previous state: Paid
New state: Cancelled

Reason: Customer return
Related return: RET-88
```

Audit records should be protected from ordinary users.

An audit trail is part of the business system, not merely a debugging log.

23. RBAC

Tawala uses **Role-Based Access Control (RBAC)**.

A user receives permissions through roles.

Example:

```
Cashier
|
+-- Create Sale
+-- View Sales
+-- Receive Payment
```

Another role:

```
Branch Manager
|
+-- Create Sale
+-- View Sales
+-- Refund
+-- View Inventory
+-- Manage Staff
+-- View Reports
```

Another:

```
Accountant
|
+-- View Transactions
+-- Manage Expenses
+-- View Financial Reports
+-- Manage Accounts
```

Roles are not enough by themselves.

Permissions must also respect business scope.

24. Permission Scope

A user may have permission to perform an action but only within a specific part of the business.

Example:

```
Jane
Role: Branch Manager
Scope: Embu Branch
```

Jane may be allowed to:

View Embu sales	YES
View Nairobi sales	NO
Edit Embu inventory	YES
Edit Nairobi stock	NO

Therefore authorization considers both:

```
Permission
+
Scope
```

This becomes a fundamental part of the security model.

25. Total Tenant Isolation

Tawala must enforce strict separation between businesses.

A request must be evaluated in the context of:

```
User
+
Business
+
Role
+
Permission
+
Scope
+
Resource
```

The system must never trust a client-provided business ID by itself.

The server must establish the authorized business context.

Isolation should exist at multiple levels:

```
Authentication
|
Business context
|
Authorization
|
Resource access
|
Database query
|
Audit
```

A bug in one layer should not automatically expose another tenant's data.

26. Business Rules

The Core should protect important business rules.

For example:

A user cannot access another business.

A cashier cannot perform an action they are not authorized to perform.

A stock transfer must have a source and destination.

A completed transaction cannot simply disappear.

A financial record should not be silently changed.

A refund must be associated with the appropriate original transaction.

These rules are more important than the database structure used to implement them.

27. State and Lifecycle

Business objects often move through states.

Example invoice:

Draft
|
Issued
|
Paid

Another possibility:

```
Draft
|
Issued
|
Partially Paid
|
Paid
```

A cancelled document should not simply be deleted.

The system should preserve the fact that it existed and was cancelled.

This improves:

- Auditability
- Reporting
- Financial correctness
- Troubleshooting

28. Configuration

Businesses should be configurable.

Examples:

```
Currency
Tax rules
Invoice numbering
Receipt numbering
Payment methods
Branches
Roles
Inventory policies
Credit policies
Business terminology
```

Configuration belongs to the business context.

A pharmacy may configure different inventory behavior from a kinyozi.

29. Extensibility

Tawala must be extensible.

The Core should provide stable foundations while specialized domains add additional capabilities.

Example:

```
TAWALA CORE
|
+-- Commerce
+-- CRM
+-- Inventory
+-- Payments
+-- Accounting
|
+-- Pharmacy Extension
|   +-- Batch Tracking
|   +-- Expiry
|
+-- Service Extension
|   +-- Appointments
|   +-- Scheduling
|
+-- SACCO Extension
    +-- Membership
    +-- Loans
    +-- Contributions
```

The exact module structure will be determined later.

The principle is:

Specialized business rules should not pollute the generic Core.

30. Commerce

Commerce may include:

```
Quotes
Sales Orders
Invoices
```

Sales
Receipts
Returns
Refunds

A typical flow might be:

Quote
↓
Sales Order
↓
Invoice
↓
Payment
↓
Receipt

Not every business needs every step.

The system should allow simpler workflows.

For example:

Customer
↓
Sale
↓
Payment
↓
Receipt

31. Inventory

Inventory should support:

Stock
Stock Movements
Stock Transfers
Stock Adjustments

Stock Counts
Reservations

Stock should be changed through meaningful business actions.

For example:

Purchase Received
↓
Stock increases

Sale Completed
↓
Stock decreases

Transfer
↓
Source decreases
Destination increases

This gives Tawala a history of why stock changed.

32. Purchasing

Purchasing may include:

Suppliers
Purchase Orders
Goods Received
Supplier Invoices
Supplier Payments
Supplier Balances

A purchase should be capable of connecting:

Supplier
+
Products
+
Inventory
+

```
Money
+
Accounting
```

This creates a connected business process instead of isolated records.

33. CRM

CRM should provide a complete relationship view.

A customer profile may eventually show:

```
Customer
|
+-- Contact information
+-- Purchase history
+-- Invoices
+-- Payments
+-- Credit
+-- Loyalty
+-- Notes
+-- Communications
+-- Follow-ups
+-- Appointments
```

CRM should therefore be integrated with the rest of Tawala.

It should not become a separate database of duplicate customers.

34. Reporting

Reporting should consume the same business truth used by operational modules.

Examples:

```
Sales today
Sales by branch
Sales by product
Sales by cashier
Sales by customer
```

```
Stock value
Low-stock products
Outstanding customer credit
Supplier balances
Expenses
Profit
Cash flow
```

Reports should not invent their own definitions of business events.

For example, "sales" should have a clear definition throughout Tawala.

35. Desktop Hardware

Desktop may provide hardware integration capabilities.

Examples:

```
Barcode Scanner
Receipt Printer
Cash Drawer
Label Printer
Customer Display
Weighing Scale
```

The architecture should isolate hardware-specific code from business logic.

Example:

```
Hardware
  ↓
Desktop Adapter
  ↓
Tawala Client
  ↓
Tawala API
  ↓
Business Domain
```

The Core should never need to know what brand of barcode scanner is attached.

36. Offline Operation

Offline operation is a potential major capability for Desktop.

However, offline support should not force the entire platform into unnecessary complexity before it is needed.

The architecture should be designed so that offline capability can be introduced safely.

Potential future architecture:

```
Desktop
  |
Local Store
  |
Sync Engine
  |
Tawala Platform
```

The exact synchronization strategy should be designed separately.

Offline mode must eventually deal with:

- Connectivity loss
- Local transactions
- Synchronization
- Conflicts
- Duplicate operations
- Security
- Recovery

This should be treated as a serious architectural subsystem.

37. API Boundary

Clients should communicate through well-defined APIs.

```
Desktop ┌
Web ───┴──> Tawala API ─> Core + Domain Modules
Mobile ─┘
```

The clients should not directly manipulate the production database.

The API should enforce:

- Authentication
 - Authorization
 - Tenant context
 - Validation
 - Business rules
 - Audit
 - Transaction boundaries
-

38. Database Principle

The database is a persistence mechanism for Tawala's business state.

It is not the architecture itself.

The architecture should exist above the database.

This means we should avoid designing the entire business model around:

`"Whatever tables we already have."`

Instead:

```
Business concept
  ↓
Domain model
  ↓
Business rules
  ↓
Persistence model
```

The database should support the domain rather than dictate it.

39. Modular Architecture

Tawala should initially favor a **modular architecture**.

The first implementation does not need dozens of independent services.

A possible structure:

```
Tawala
|
+-- Core
|
+-- Sales
|
+-- Inventory
|
+-- Purchasing
|
+-- CRM
|
+-- Accounting
|
+-- Payments
|
+-- Reporting
|
+-- Specialized Domains
```

Each module should have a clear responsibility.

Modules should communicate through explicit contracts rather than reaching into each other's internal implementation.

40. Avoid Premature Microservices

Tawala does not need distributed infrastructure simply because it is large in concept.

A modular monolith can provide:

- Strong consistency
- Simple deployment
- Easier development
- Easier debugging
- Clear domain boundaries
- Lower operational complexity

Individual components can be extracted later if real requirements justify it.

The architecture should make extraction possible without requiring extraction immediately.

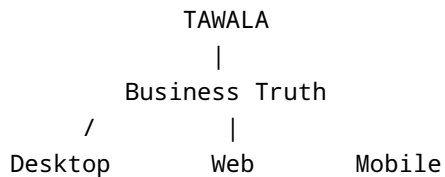
41. Source of Truth

There should be one authoritative business system.

Not:

```
Desktop database = truth
Web database = truth
Mobile database = truth
```

Instead:



Local storage may exist for performance or offline operation.

It must not silently create competing versions of the business.

42. Security Principles

Security should be designed into the platform.

Core principles:

1. Every request has an authenticated identity.
 2. Every business operation runs inside an authorized business context.
 3. Permissions are explicit.
 4. Access is scoped.
 5. Tenant boundaries are enforced server-side.
 6. Clients cannot grant themselves permissions.
 7. Sensitive actions are audited.
 8. Financial and business records are protected from silent modification.
 9. Secrets are never stored in client applications unnecessarily.
 10. Integrations use controlled credentials and permissions.
-

43. The Core Must Stay Small

This is one of the most important rules.

The Core should not become a dumping ground.

Before adding something to Core, ask:

Is this concept genuinely useful across multiple domains?

If yes:

Candidate for Core

If no:

Candidate for specialized module

For example:

Party	→ Core
Product	→ Core
Location	→ Core
Payment	→ Core
Prescription	→ Pharmacy domain
Appointment	→ Service domain
Loan	→ Finance/SACCO domain

The Core should remain understandable.

44. Architecture Decision: Generic First

Tawala should prefer:

Generic capability
+
Configuration

+
Specialized extension

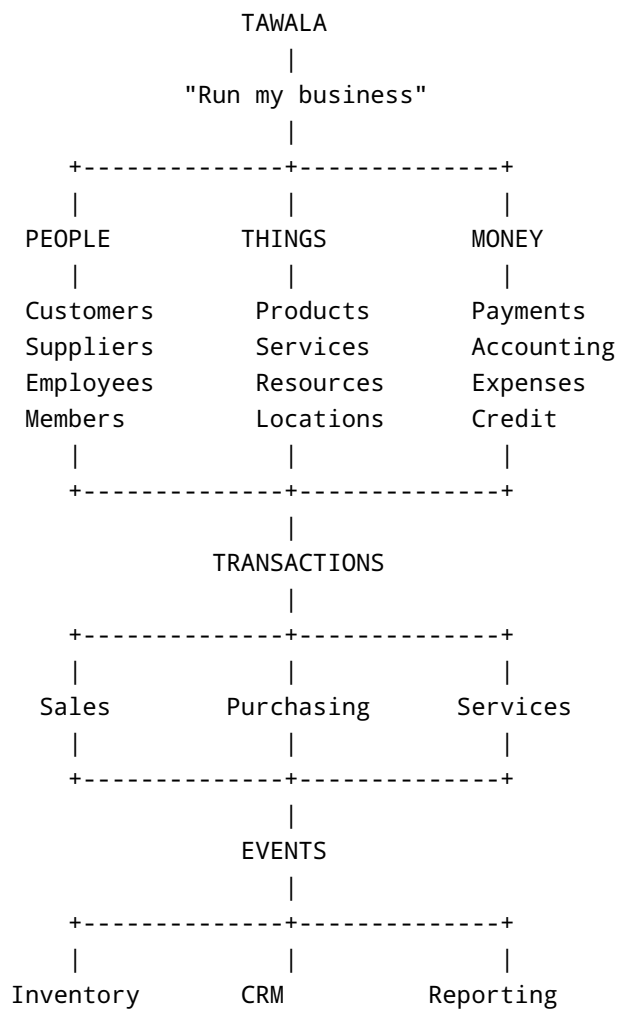
over:

Industry-specific Core logic

This allows Tawala to grow across industries without constantly redesigning its foundation.

45. The Tawala Mental Model

The easiest way to understand the architecture is:



This is the conceptual heart of Tawala.

46. The Three-Layer Model

Tawala can be understood as three major layers.

Layer 1 — Experience

How people interact with Tawala.

Desktop
Web
Mobile

Layer 2 — Business

What the business actually does.

Sales
Inventory
CRM
Purchasing
Accounting
Payments
Specialized domains

Layer 3 — Core

The shared foundation.

Identity
Tenancy
Organization
RBAC
Parties
Products
Locations

Resources
Documents
Transactions
Events
Audit
Configuration

The layers should remain clearly separated.

47. What the Core Does NOT Know

The Core should not care:

Whether the user is on Android or Windows.

Whether the business is a pharmacy or kinyozi.

Whether a barcode scanner is connected.

Whether the request came from Web or Desktop.

Whether the business has one branch or fifty.

Whether the customer prefers M-PESA or cash.

The Core provides the underlying capabilities and guarantees.

The surrounding domains and clients provide the context.

48. What Tawala Should Ultimately Feel Like

A business owner should not need to think:

"I am using a POS."

or:

"I am using a CRM."

or:

"I am using an accounting system."

They should feel:

"This is where I run my business."

That is the product goal.

49. Architectural North Star

The entire architecture can be summarized as:

```
ONE PLATFORM
|
+-- MANY BUSINESSES
|   |
|   +-- MANY INDUSTRIES
|   |
|   +-- MANY BRANCHES
|   |
|   +-- MANY USERS
|
+-- MANY CAPABILITIES
|
+-- ONE BUSINESS MODEL
|
+-- ONE SOURCE OF TRUTH
|
+-- MULTIPLE CLIENTS
    |
    +-- Desktop
    +-- Web
    +-- Mobile
```

The most important rule:

Different businesses may use Tawala differently, but they should all be operating on the same coherent platform.

50. What Comes Next

This document is intentionally **not finished architecture**.

It is the foundation for deeper design.

The next stage is to challenge every major concept.

We should now investigate:

```
Business
User
Membership
Role
Permission
Party
Product
Service
Resource
Location
Document
Transaction
Payment
Account
Event
Audit
```

For each one, we will ask:

1. What does it mean?
2. Why does it exist?
3. What does it own?
4. What can change?
5. What relationships does it have?
6. What rules govern it?
7. Is it truly Core?
8. Can it work across multiple industries?
9. Can it work across Desktop, Web and Mobile?
10. What happens when it is deleted, cancelled, transferred, or changed?

Only after those questions are answered should the actual database and API contracts be designed.

51. Clean-Room Architecture Rule

This architecture is intentionally independent of the existing Tawala implementation.

The existing implementation must not be used to justify architectural decisions at this stage.

After the Core architecture is reviewed and stabilized, the existing Tawala implementation will be examined separately.

The comparison will determine whether each existing component is:

KEEP

ADAPT / REFACTOR

or

REPLACE / MIGRATE

A V2 will only be justified if the existing architecture cannot reasonably evolve toward the Core.

52. Final Principle

Tawala should not be engineered around today's screens.

It should not be engineered around today's database.

It should not be engineered around today's framework.

It should be engineered around **the business system Tawala is intended to become.**

The technology can change.

The interfaces can change.

The modules can evolve.

The infrastructure can change.

But the underlying business model should remain coherent.

Tawala Core is the foundation that makes all of those changes possible without changing what Tawala fundamentally is.

END OF TAWALA_CORE_ARCHITECTURE.md